$$\tilde{\mathbf{K}} \cdot \mathbf{f} = \sigma \mathbf{f} \tag{19.1.7}$$

which we can solve with any convenient matrix eigenvalue routine (see Chapter 11). Note that if our original problem had a symmetric kernel, then the matrix $\mathbf{K}$ would be symmetric. However, since the weights w_j are not equal for most quadrature rules, the matrix $\tilde{\mathbf{K}}$ (equation 19.1.5) is not symmetric. The matrix eigenvalue problem is much easier for symmetric matrices, and so we should restore the symmetry if possible. Provided the weights are positive (which they are for Gaussian quadrature), we can define the diagonal matrix $\mathbf{D} = \text{diag}(w_j)$ and its square root, $\mathbf{D}^{1/2} = \text{diag}(\sqrt{w_j})$. Then equation (19.1.7) becomes

$$\mathbf{K} \cdot \mathbf{D} \cdot \mathbf{f} = \sigma \mathbf{f}$$

Multiplying by $\mathbf{D}^{1/2}$, we get

$$\left(\mathbf{D}^{1/2} \cdot \mathbf{K} \cdot \mathbf{D}^{1/2}\right) \cdot \mathbf{h} = \sigma \mathbf{h} \tag{19.1.8}$$

where $\mathbf{h} = \mathbf{D}^{1/2} \cdot \mathbf{f}$. Equation (19.1.8) is now in the form of a symmetric eigenvalue problem.

Solution of equations (19.1.7) or (19.1.8) will in general give N eigenvalues, where N is the number of quadrature points used. For square-integrable kernels, these will provide good approximations to the lowest N eigenvalues of the integral equation. Kernels of *finite rank* (also called *degenerate* or *separable* kernels) have only a finite number of nonzero eigenvalues (possibly none). You can diagnose this situation by a cluster of eigenvalues σ that are zero to machine precision. The number of nonzero eigenvalues will stay constant as you increase N to improve their accuracy. Some care is required here: A nondegenerate kernel can have an infinite number of eigenvalues that have an accumulation point at $\sigma = 0$. You distinguish the two cases by the behavior of the solution as you increase N. If you suspect a degenerate kernel, you will usually be able to solve the problem by analytic techniques described in all the textbooks.

CITED REFERENCES AND FURTHER READING:

Delves, L.M., and Mohamed, J.L. 1985, *Computational Methods for Integral Equations* (Cambridge, UK: Cambridge University Press).[1]

Atkinson, K.E. 1976, *A Survey of Numerical Methods for the Solution of Fredholm Integral Equations of the Second Kind* (Philadelphia: S.I.A.M.).

19.2 Volterra Equations

Let us now turn to Volterra equations, of which our prototype is the Volterra equation of the second kind,

$$f(t) = \int_a^t K(t,s) f(s)\, ds + g(t) \tag{19.2.1}$$

Most algorithms for Volterra equations march out from $t = a$, building up the solution as they go. In this sense they resemble not only forward substitution (as discussed in §19.0), but also initial value problems for ordinary differential equations. In fact, many algorithms for ODEs have counterparts for Volterra equations.

The simplest way to proceed is to solve the equation on a mesh with uniform spacing:

$$t_i = a + ih, \quad i = 0, 1, \ldots, N, \qquad h \equiv \frac{b-a}{N} \tag{19.2.2}$$

To do so, we must choose a quadrature rule. For a uniform mesh, the simplest scheme is the trapezoidal rule, equation (4.1.11):

$$\int_a^{t_i} K(t_i, s) f(s)\, ds = h\left(\tfrac{1}{2}K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j + \tfrac{1}{2}K_{ii} f_i\right) \tag{19.2.3}$$

Thus the trapezoidal method for equation (19.2.1) is

$$\begin{aligned} f_0 &= g_0 \\ (1 - \tfrac{1}{2}hK_{ii})\, f_i &= h\left(\tfrac{1}{2}K_{i0} f_0 + \sum_{j=1}^{i-1} K_{ij} f_j\right) + g_i, \qquad i = 1, \ldots, N \end{aligned} \tag{19.2.4}$$

(For a Volterra equation of the first kind, the leading 1 on the left would be absent, and g would have opposite sign, with corresponding straightforward changes in the rest of the discussion.)

Equation (19.2.4) is an explicit prescription that gives the solution in $O(N^2)$ operations. Unlike Fredholm equations, it is not necessary to solve a system of linear equations. Volterra equations thus usually involve less work than the corresponding Fredholm equations, which, as we have seen, do involve the inversion of, sometimes large, linear systems.

The efficiency of solving Volterra equations is somewhat counterbalanced by the fact that *systems* of these equations occur more frequently in practice. If we interpret equation (19.2.1) as a *vector* equation for the vector of m functions $f(t)$, then the kernel $K(t, s)$ is an $m \times m$ matrix. Equation (19.2.4) must now also be understood as a vector equation. For each i, we have to solve the $m \times m$ set of linear algebraic equations by Gaussian elimination.

The routine `voltra` below implements this algorithm. You must supply an external function or functor that returns the kth function of the vector $g(t)$ at the point t and another that returns the (k, l) element of the matrix $K(t, s)$ at (t, s). The routine `voltra` then returns the vector $f(t)$ at the regularly spaced points t_i.

voltra.h

```
template <class G, class K>
void voltra(const Doub t0, const Doub h, G &g, K &ak, VecDoub_O &t, MatDoub_O &f)
```

Solves a set of `m` linear Volterra equations of the second kind using the extended trapezoidal rule. On input, `t0` is the starting point of the integration and `h` is the stepsize. `g(k,t)` is a user-supplied function or functor that returns $g_k(t)$, while `ak(k,l,t,s)` is another user-supplied function or functor that returns the (k, l) element of the matrix $K(t, s)$. The solution is returned in `f[0..m-1][0..n-1]`, with the corresponding abscissas in `t[0..n-1]`, where `n-1` is the number of steps to be taken. The value of `m` is determined from the row-dimension of the solution matrix `f`.

```
{
    Int m=f.nrows();
```

```
    Int n=f.ncols();
    VecDoub b(m);
    MatDoub a(m,m);
    t[0]=t0;
    for (Int k=0;k<m;k++) f[k][0]=g(k,t[0]);          Initialize.
    for (Int i=1;i<n;i++) {                           Take a step h.
        t[i]=t[i-1]+h;
        for (Int k=0;k<m;k++) {
            Doub sum=g(k,t[i]);                       Accumulate right-hand side of linear
            for (Int l=0;l<m;l++) {                       equations in sum.
                sum += 0.5*h*ak(k,l,t[i],t[0])*f[l][0];
                for (Int j=1;j<i;j++)
                    sum += h*ak(k,l,t[i],t[j])*f[l][j];
                if (k == l)                           Left-hand side goes in matrix a.
                    a[k][l]=1.0-0.5*h*ak(k,l,t[i],t[i]);
                else
                    a[k][l] = -0.5*h*ak(k,l,t[i],t[i]);
            }
            b[k]=sum;
        }
        LUdcmp alu(a);                                Solve linear equations.
        alu.solve(b,b);
        for (Int k=0;k<m;k++) f[k][i]=b[k];
    }
}
```

For nonlinear Volterra equations, equation (19.2.4) holds with the product $K_{ij} f_j$ replaced by $K_{ij}(f_j)$, and similarly for the other two products of K's and f's. Thus, for each i we solve a nonlinear equation for f_i with a known right-hand side. Newton's method (§9.4 or §9.6) with an initial guess of f_{i-1} usually works very well provided the stepsize is not too big.

Higher-order methods for solving Volterra equations are, in our opinion, not as important as for Fredholm equations, since Volterra equations are relatively easy to solve. However, there is an extensive literature on the subject. Several difficulties arise. First, any method that achieves higher order by operating on several quadrature points simultaneously will need a special method to get started, when values at the first few points are not yet known.

Second, stable quadrature rules can give rise to unexpected instabilities in integral equations. For example, suppose we try to replace the trapezoidal rule in the algorithm above with Simpson's rule. Simpson's rule naturally integrates over an interval $2h$, so we easily get the function values at the even mesh points. For the odd mesh points, we could try appending one panel of the trapezoidal rule. But to which end of the integration should we append it? We could do one step of the trapezoidal rule followed by all Simpson's rule, or Simpson's rule with one step of the trapezoidal rule at the end. Surprisingly, the former scheme is unstable, while the latter is fine!

A simple approach that can be used with the trapezoidal method given above is Richardson extrapolation: Compute the solution with stepsizes h and $h/2$. Then, assuming the error scales with h^2, compute

$$f_{\mathrm{E}} = \frac{4f(h/2) - f(h)}{3} \tag{19.2.5}$$

This procedure can be repeated as with Romberg integration.

The general consensus is that the best of the higher-order methods is the *block-by-block method* (see [1]). Another important topic is the use of variable stepsize